

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA0302	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	B. Mohith Manoj	Reg. No.:	192472018
Section / Batch:	CSE - AI	Date:	27/07/2026

Code of Conduct

I T. Dharani Devi (192425036) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B. Mohith Manoj

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
 - States
 - Input alphabet
 - Transition table
 - Initial state
 - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

$q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$

Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions**3. Problem Statement**

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search

Prefix Search

Suffix Search

Date Search

Phone Number Search

Hashtag Search

Mention (@username) Search

Example Input

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

User Menu

1.Search Date

2.Search Phone Number

3.Search Hashtag

4.Search Mention

5.Search Prefix

6.Search Suffix

Expected Output

Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Experiment 1: User Registration Validation using Regular Expressions

Aim:

To develop a Python application using Regular Expressions (Regex) to validate user credentials such as Email Address, Password, and Mobile Number according to predefined security rules.

Procedure:

1. Import the re (Regular Expression) module.
2. Read the Email Address, Password, and Mobile Number from the user.
3. Validate the Email Address using the specified Regex pattern.
4. Validate the Password by checking:
 - o Minimum 8 characters
 - o At least one uppercase letter
 - o At least one lowercase letter
 - o At least one digit
 - o At least one special character (@, #, \$, %, &, !)
5. Validate the Mobile Number to ensure it has exactly 10 digits and starts with a digit between 6 and 9.
6. Display whether each input is valid or invalid.

Program:

```
import re

def validate_email(email):
    pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
    if re.fullmatch(pattern, email):
        print("Valid Email")
    else:
        print("Invalid Email")

def validate_password(password):
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!])[A-Za-z\d@#%&!]{8,}$'
    if re.fullmatch(pattern, password):
        print("Strong Password")
    else:
        print("Weak Password")

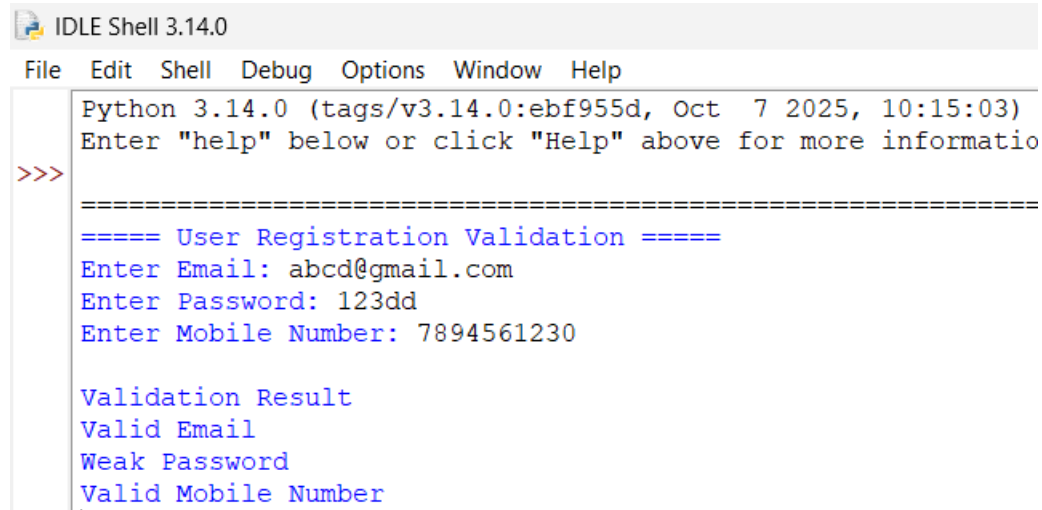
def validate_mobile(mobile):
    pattern = r'^[6-9]\d{9}$'
    if re.fullmatch(pattern, mobile):
        print("Valid Mobile Number")
    else:
        print("Invalid Mobile Number")

print("==== User Registration Validation =====")
```

```
email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

print("\nValidation Result")
validate_email(email)
validate_password(password)
validate_mobile(mobile)
```

Output:



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03)
Enter "help" below or click "Help" above for more informatio
>>>
=====
==== User Registration Validation ====
Enter Email: abcd@gmail.com
Enter Password: 123dd
Enter Mobile Number: 7894561230

Validation Result
Valid Email
Weak Password
Valid Mobile Number
```

Result:

Thus, the Python program was successfully implemented to validate Email Address, Password, and Mobile Number using Regular Expressions.

Experiment 2: Deterministic Finite Automata (DFA) Simulator

Aim:

To develop a Python-based Deterministic Finite Automata (DFA) simulator that accepts multiple input strings, simulates state transitions, and determines whether each string is accepted or rejected.

Procedure:

1. Define the DFA states and transition table.
2. Specify the initial state and final (accepting) state.
3. Read the number of input strings.
4. Accept each input string from the user.
5. Process the string character by character according to the transition table.
6. Display the transition path.
7. Check whether the final state is an accepting state.
8. Display Accepted or Rejected.

Program:

```
# DFA Simulator
```

```
# Language: Strings ending with "ab"
```

```
dfa = {  
    ('q0', 'a'): 'q1',  
    ('q0', 'b'): 'q0',  
  
    ('q1', 'a'): 'q1',  
    ('q1', 'b'): 'q2',  
  
    ('q2', 'a'): 'q1',  
    ('q2', 'b'): 'q0'  
}
```

```
initial_state = 'q0'  
final_states = ['q2']
```

```
n = int(input("Enter number of input strings: "))
```

```
for i in range(n):
```

```
    string = input("\nEnter String: ")
```

```
    current = initial_state
```

```
    path = [current]
```

```
    valid = True
```

```

for ch in string:

    if (current, ch) in dfa:

        current = dfa[(current, ch)]
        path.append(current)

    else:

        valid = False
        break

print("Transition Path:")

print(" -> ".join(path))

if valid and current in final_states:

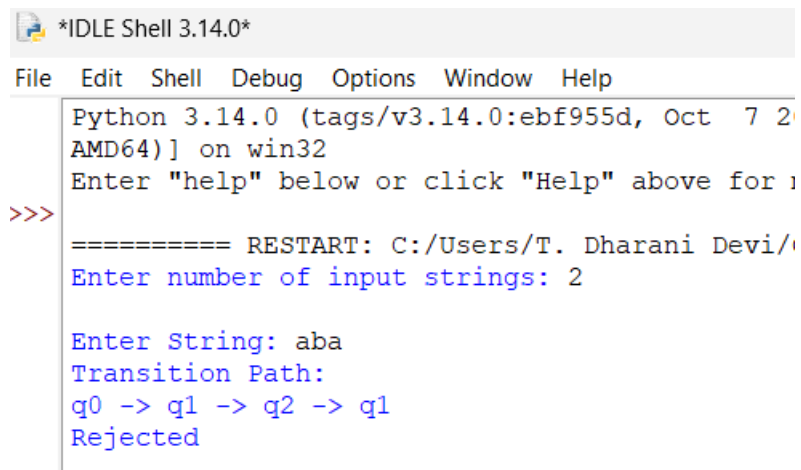
    print("Accepted")

else:

    print("Rejected")

```

Output:



```

*IDLE Shell 3.14.0*
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2024, AMD64) on win32
Enter "help" below or click "Help" above for more
>>>
===== RESTART: C:/Users/T. Dharani Devi/Python314/IDLE Shell 3.14.0
Enter number of input strings: 2

Enter String: aba
Transition Path:
q0 -> q1 -> q2 -> q1
Rejected

```

Result:

Thus, the Python program successfully simulated a Deterministic Finite Automata (DFA) and correctly determined whether the given input strings were accepted or rejected.

Experiment 3: Smart Pattern Matching Engine using Regular Expressions

Aim:

To develop a Python-based Smart Pattern Matching Engine using Regular Expressions to search for dates, phone numbers, hashtags, mentions, prefixes, and suffixes in a given text.

Procedure:

1. Import the re module.
2. Store the sample text containing dates, phone numbers, hashtags, mentions, and words.
3. Display a menu of search options.
4. Accept the user's choice.
5. Apply the appropriate Regular Expression pattern based on the selected option.
6. Display all matching patterns found in the text.
7. Repeat until the user chooses to exit.

Program:

```
import re

text = """
Meeting on 12/09/2026
Project Deadline: 25/12/2026
Call 9876543210
Office Number 9123456789
#NLP
#Python
@OpenAI
@ChatGPT
natural language processing
machine learning
deep learning
"""

while True:

    print("\n===== SMART PATTERN MATCHING ENGINE =====")
    print("1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":

        result = re.findall(r'\b\d{2}/\d{2}/\d{4}\b', text)
```

```
print("Dates Found:", result)

elif choice == "2":

    result = re.findall(r'\b[6-9]\d{9}\b', text)

    print("Phone Numbers:", result)

elif choice == "3":

    result = re.findall(r'#\w+', text)

    print("Hashtags:", result)

elif choice == "4":

    result = re.findall(r'@\w+', text)

    print("Mentions:", result)

elif choice == "5":

    prefix = input("Enter Prefix: ")

    pattern = r'\b' + re.escape(prefix) + r'\w*'

    result = re.findall(pattern, text, re.IGNORECASE)

    print("Words:", result)

elif choice == "6":

    suffix = input("Enter Suffix: ")

    pattern = r'\b\w*' + re.escape(suffix) + r'\b'

    result = re.findall(pattern, text, re.IGNORECASE)

    print("Words:", result)

elif choice == "7":

    print("Program Ended.")
    break

else:

    print("Invalid Choice")
```

Output:

```
===== SMART PATTERN MATCHING ENGINE =====
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter your choice: 3
Hashtags: ['#NLP', '#Python']

===== SMART PATTERN MATCHING ENGINE =====
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
```

Result:

Thus, the Python program was successfully implemented to perform pattern matching using Regular Expressions and efficiently search for dates, phone numbers, hashtags, mentions, prefixes, and suffixes within the given text.